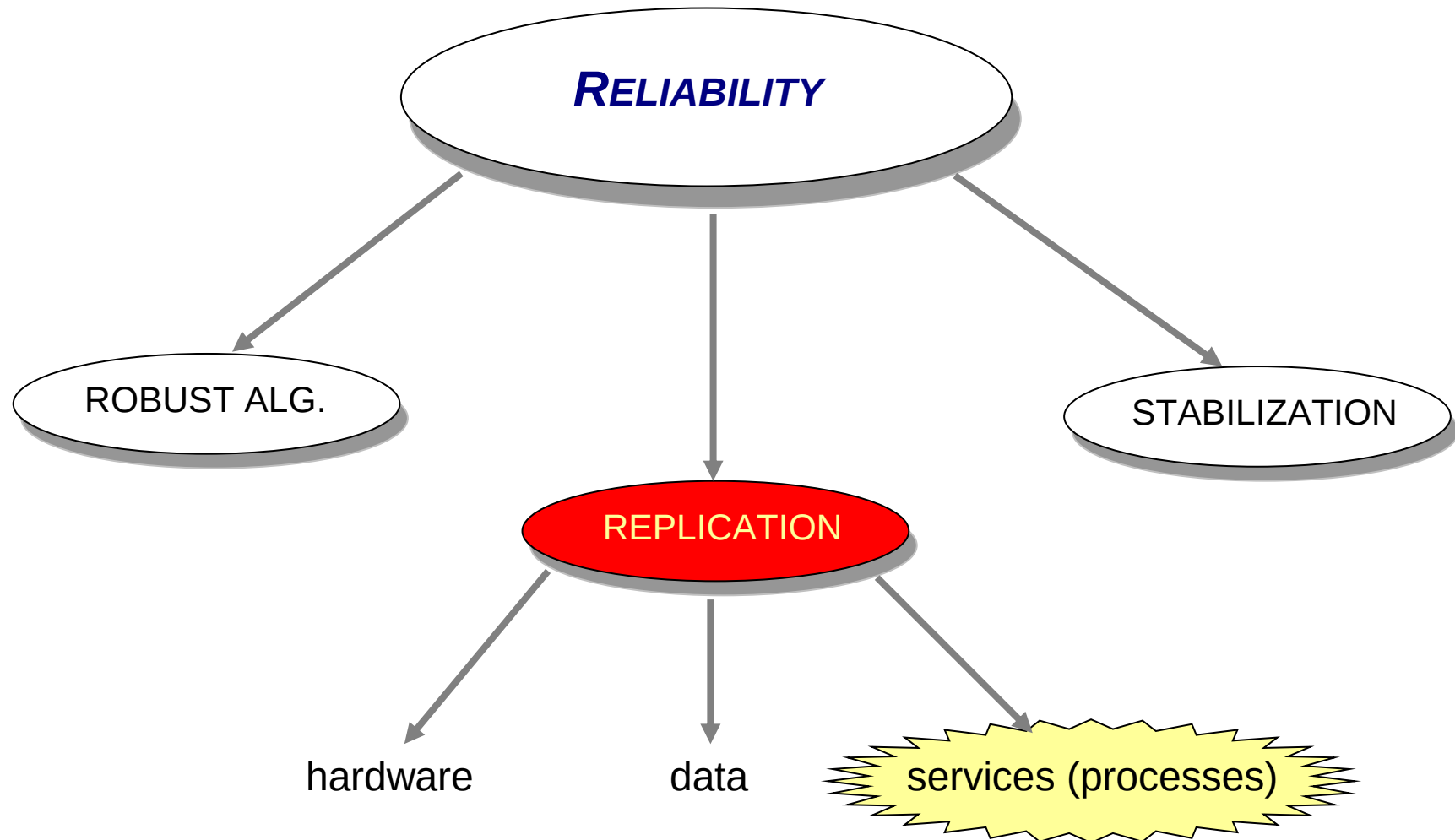


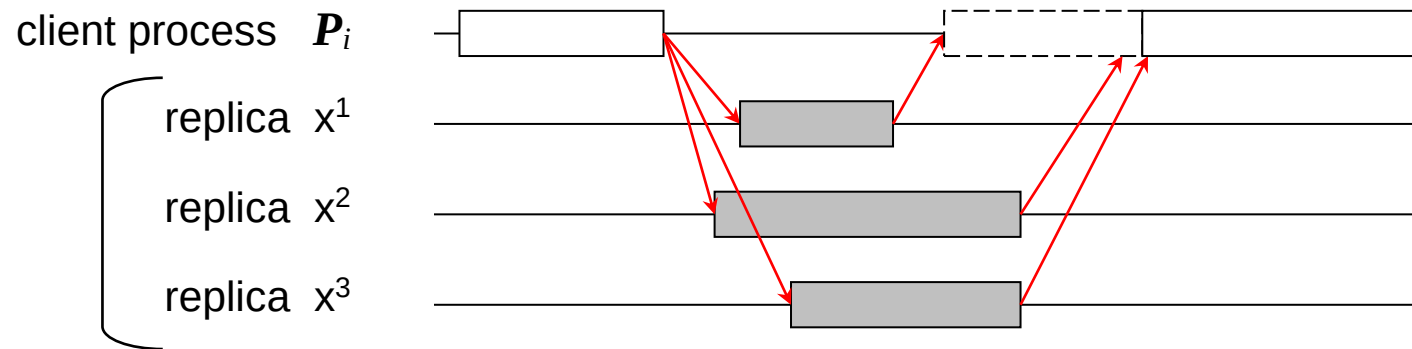


REPLICATION

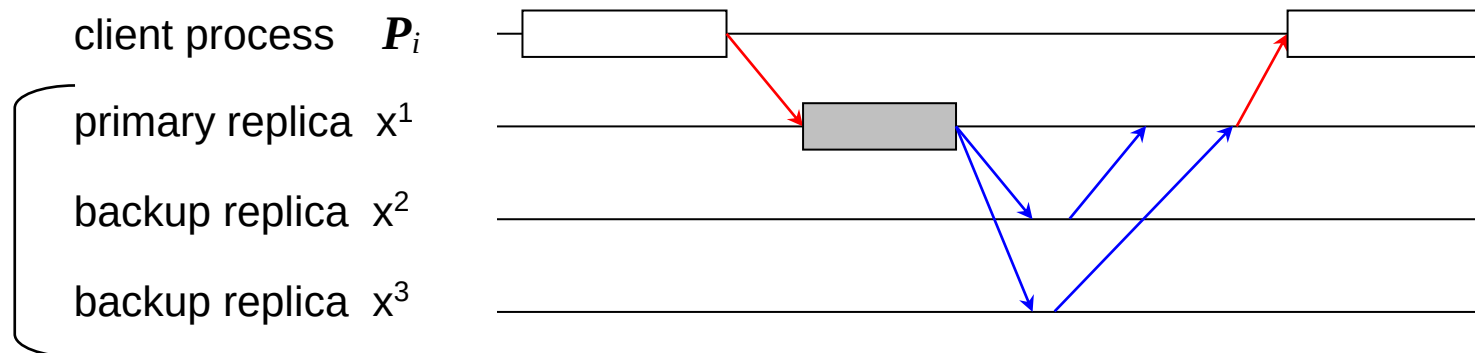


PROCESS REPLICATION

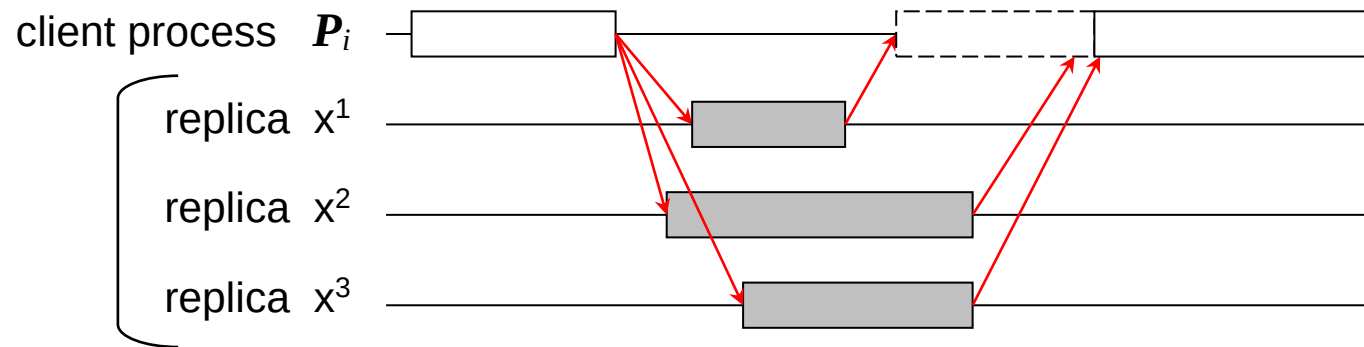
Active replication = state-machine replication:



Passive replication = primary-backup:

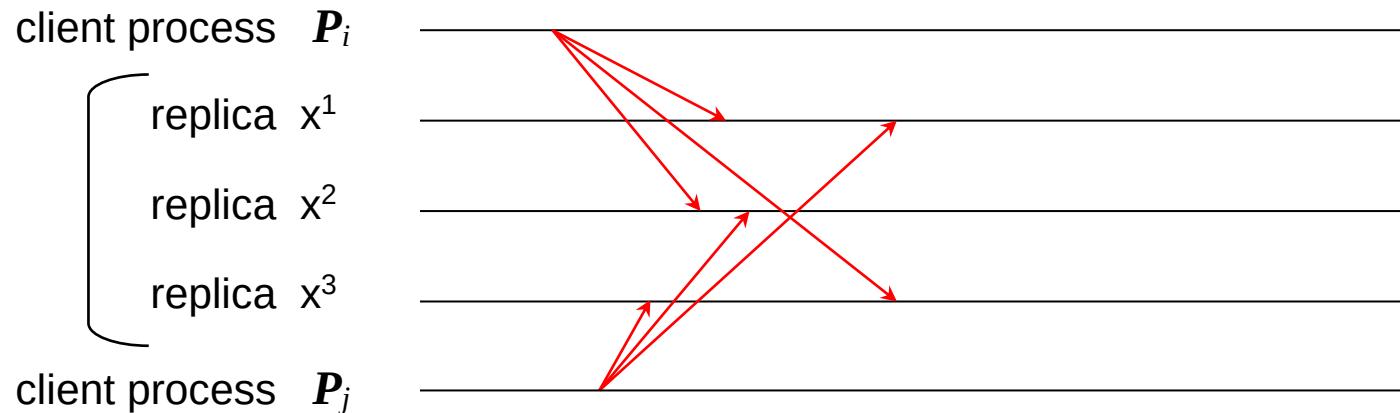


ACTIVE REPLICATION



- client may choose response gathering strategy: first come, all existing (n), $R < n$
- may deal with Byzantine failures, depending on R (e.g. $R > 3f$)
- dynamic groups (yet group membership irrelevant)
- most of cloud frameworks/toolkits: Google Spanner, Apache Zookeeper, MySQL Group Replication

- consistency: **multiple** clients may issue requests simultaneously



- requests have to be **totally ordered**
- an exact ordering relation is irrelevant – all replicas just have to process requests in the same order

Nondeterminism

- nondeterministic processing not allowed – may lead to different replies to the same client request
- cannot be distinguished from Byzantine failures
- voting or Byzantine agreement doesn't solve the issue – each reply may be different
- even if client chooses one response, states of distinct replicas have evolved to inconsistency

Summary

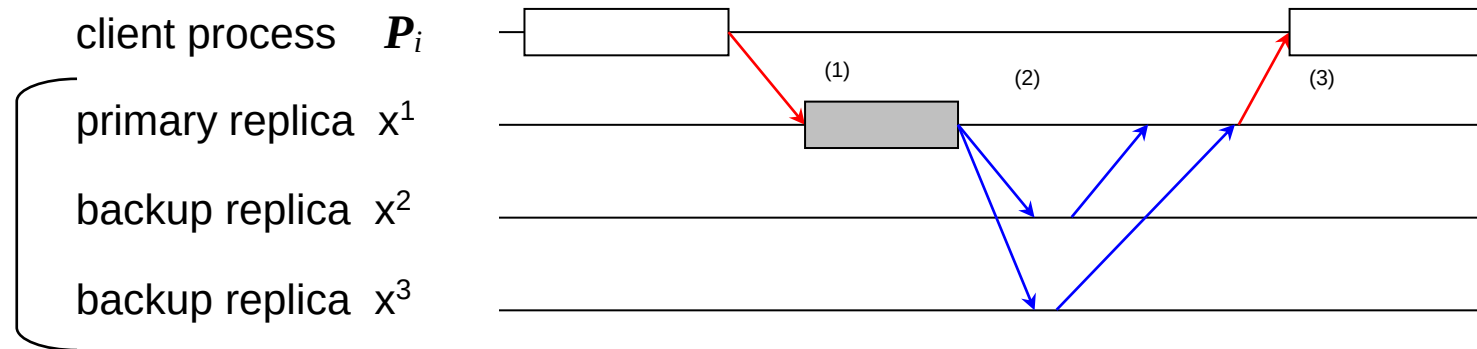
Advantages:

- client never reissues the request (if only $f < N$, obviously)
- Byzantine failures could be allowed

Disadvantages:

- only deterministic operations allowed (consistency requirement)
- larger resource utilization: $n \leq N$ replicas process each request
- non scalable: increasing N does not increase the performance (throughput)

PASSIVE REPLICATION



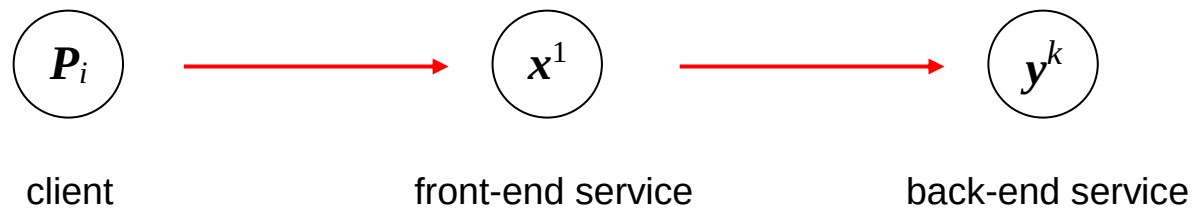
- **consistency** of replicas: the primary replica imposes total order of updates
- dynamic groups (required group **membership** service)
- **reliability** (the primary replica fails):
 - update operation must be atomic ($\rightarrow$ *Reliable Broadcast*)
 - new primary has to be elected
 - client may observe a delay
 - client may need to reissue the request
 - each request has to be uniquely identified

Nondeterminism

- nondeterministic processing is allowed since only one replica is processing requests
- and updates ensure global consistency among the group members

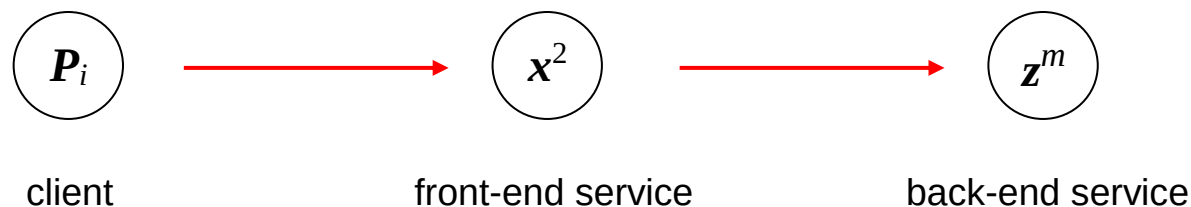
... unless ...

- ... nested invocations are considered

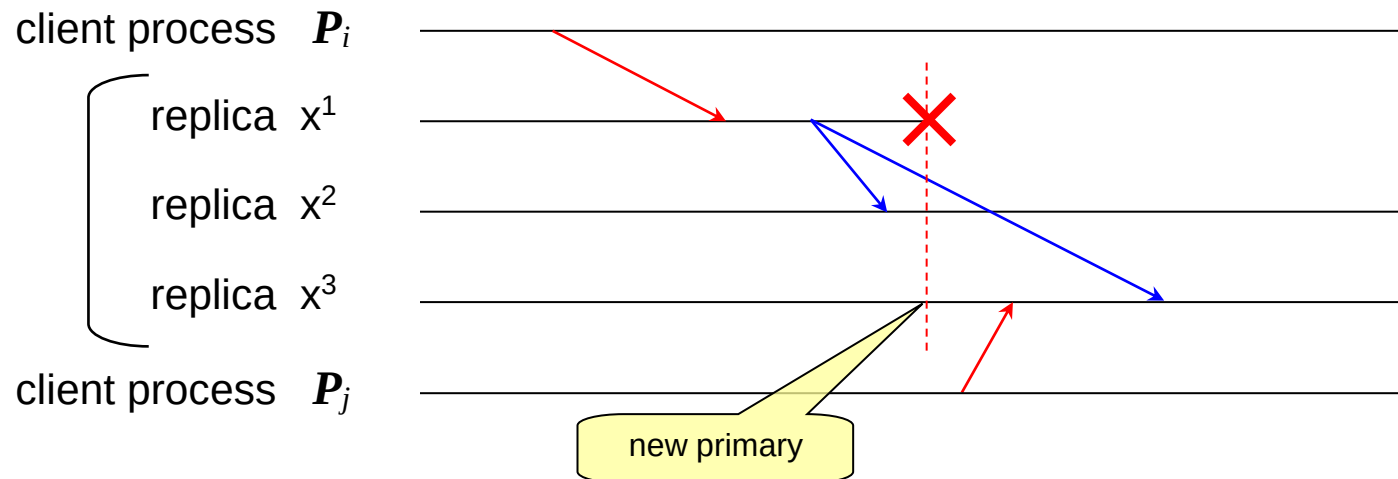


Nested invocations

- the primary x^1 invokes another server $y^{(k)}$
- based on a nondeterministic decision
- then, x^1 fails after receiving response from $y^{(k)}$
- new primary x^2 invokes another server $z^{(m)}$
- based on its own nondeterministic decision
- both y and z have been updated – inconsistent global state



- consistency: **multiple** clients complicate the issue



- requests have to be **synchronized with membership changes**

Summary

Advantages:

- low resource utilization (only one replica is processing requests)
- possible nondeterministic processing (as long as there are no nested invocations)

Disadvantages:

- no real-time apps (large delays can happen)
- no Byzantine failures (clients must not receive incorrect answers)

GROUP COMMUNICATION

Message ordering:

- active replication requires a **totally ordered multicast** – **TOcast**
(sometimes called **atomic broadcast** – **ABcast**)

Views:

- each membership change in a group (due to process failures or recovery) constitutes a new **view** of this group
- passive replication requires a **view synchronous multicast** – **VScast**



TOcast

TOcast = RBCast + TO (total order of delivery)

RBCast (*Reliable Broadcast*):

Termination: *if P_c broadcasts $m \Rightarrow$ it eventually delivers m*

Agreement: *if some P_c delivers $m \Rightarrow$ all P_c deliver m*

Validity: *m was broadcast by some P_i
 m is delivered at most once*

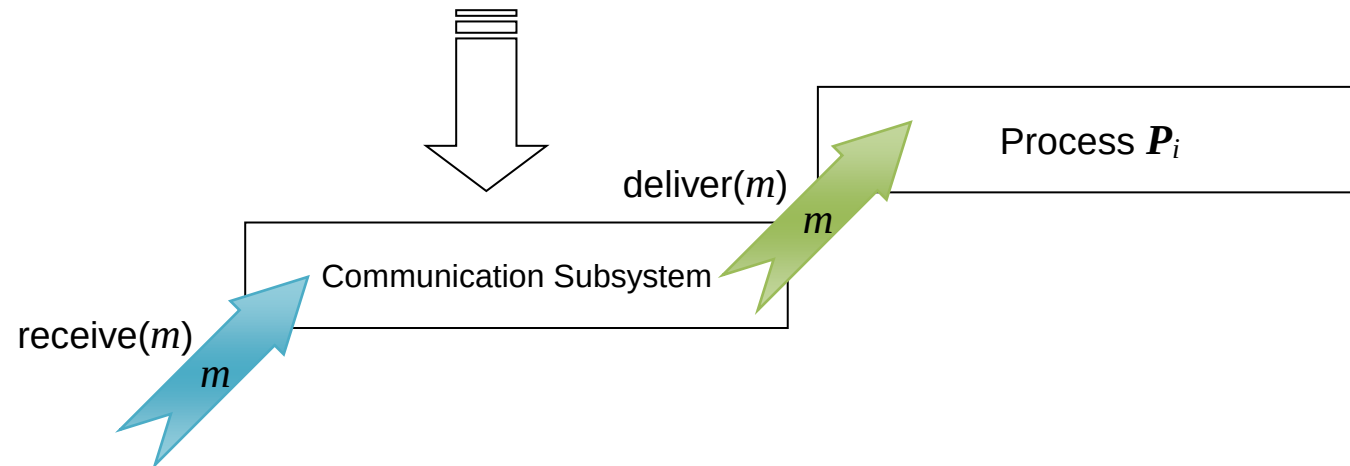
correct



RELIABLE BROADCAST BY MESSAGE DIFFUSION

C_i :

```
RBcast( $m$ ) :  
  {  
    send  $m$  to all  $P$  (including  $P_i$ )  
  }  
  
Rdeliver( $m$ ) occurs as follows:  
  {  
    when receive( $m$ ) for the first time (for that  $m$ )  
      if sender( $m$ )  $\neq P_i$  then send  $m$  to all  
  }
```



UNIFORM BROADCAST

In RBcast:

- if sender(m) has failed $\Rightarrow m$ is delivered by all P_c or by none

UBcast (*Uniform Broadcast*):

$$\left(\begin{array}{c} \cdot \\ \cdot \\ \cdot \end{array} \right)$$
[illegible]

- note: P_f can deliver whatever it wants

failed

VIEW SYNCHRONOUS MULTICAST

- consistent set of messages delivered despite group membership changes
- $\mathbf{v}_i, \mathbf{v}_{i+1}$ – two subsequent views of the same group

VScast:

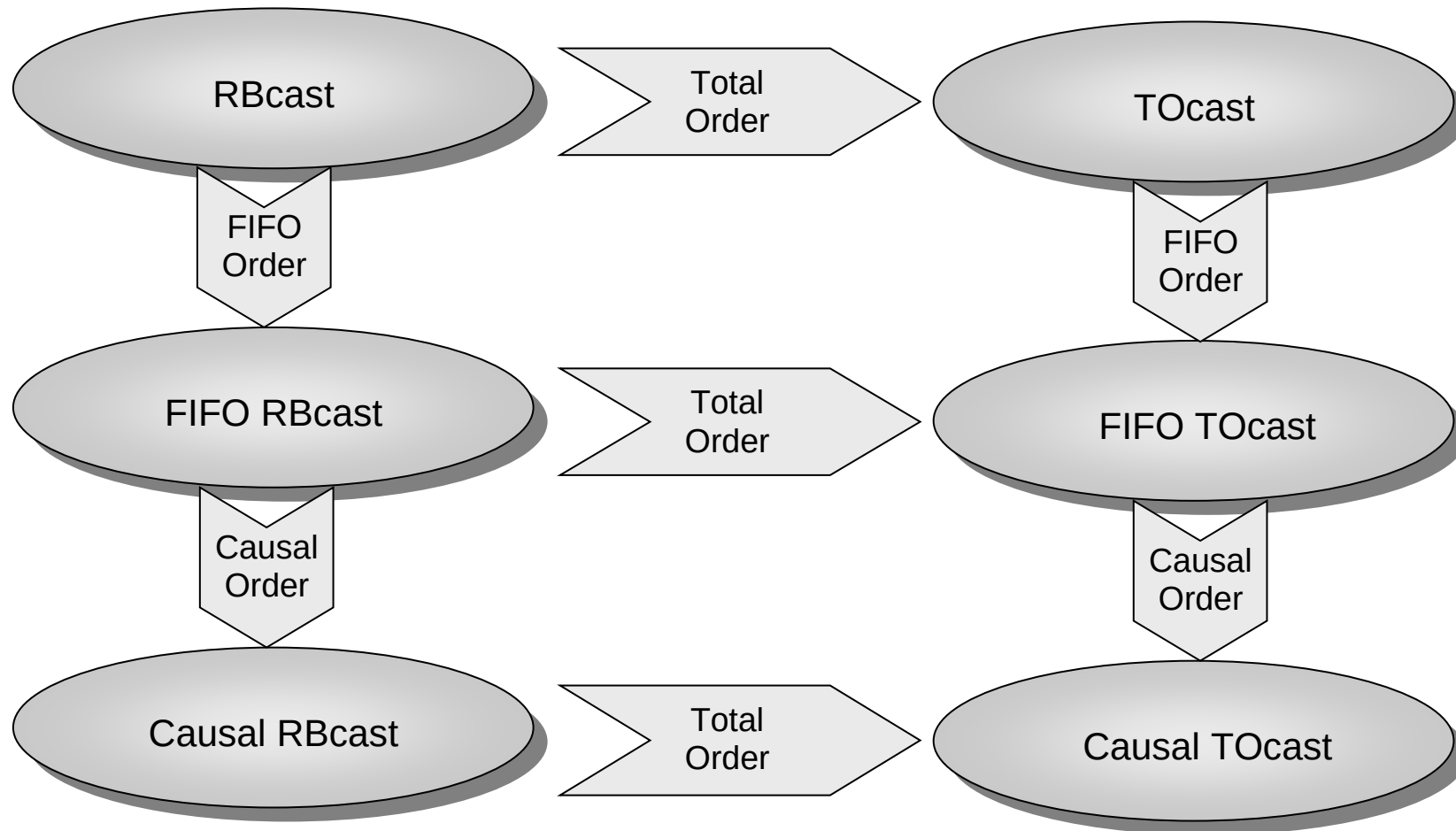
(. . .)

Agreement: *all $P_c \in (\mathbf{v}_i \cap \mathbf{v}_{i+1})$ eventually deliver m*

Validity: *if m was sent in $\mathbf{v}_i$, then it has to be delivered at each $P_c \in (\mathbf{v}_i \cap \mathbf{v}_{i+1})$ before any message from $\mathbf{v}_{i+1}$*

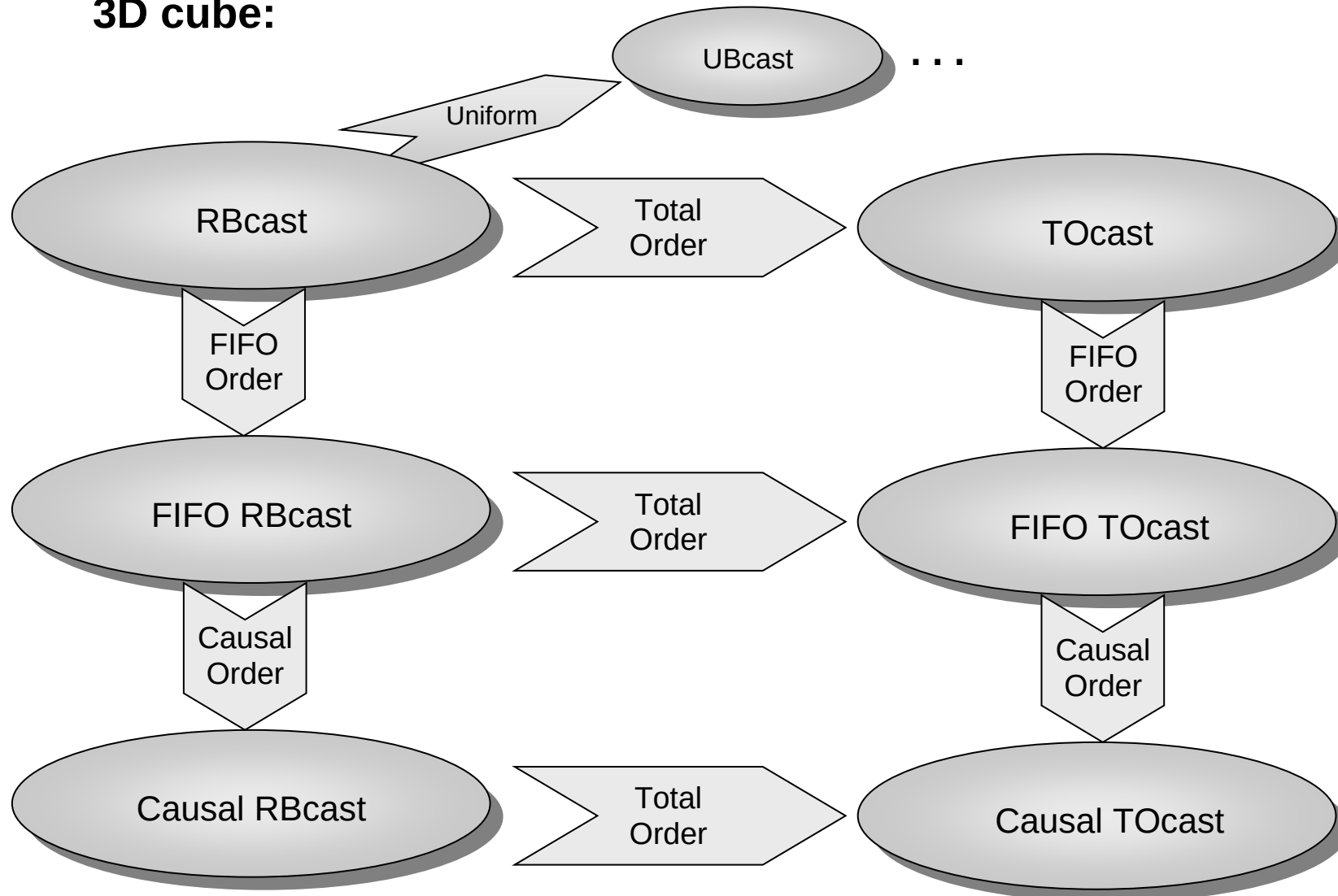


ORDERED BROADCASTS





3D cube:





CONSENSUS & TOCAST

C (Consensus):

Termination: *all P_c decides exactly 1 value v_i*

Agreement: *all P_c decides the same v_i*

Validity: *v_i was proposed by some P_i*

TOcast = RBcast + TO

RBcast:

Termination: *if P_c broadcasts $m \Rightarrow$ it eventually delivers m*

Agreement: *if some P_c delivers $m \Rightarrow$ all P_c deliver m*

Validity: *m was broadcast by some P_i
 m is delivered at most once*

Equivalence of Consensus problem with TOcast and VScast

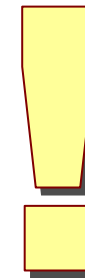
$$\text{TOcast} \cong C \cong \text{VScast}$$

What does it mean?

- if we have a solution for TOcast $\Rightarrow$ we can solve C
- if we have a solution for C $\Rightarrow$ we can solve TOcast
- the first implication is trivial, we will see the latter later

FLP'85 IMPOSSIBILITY RESULT

**There is no deterministic
1-resilient fail-stop Consensus solution
in an asynchronous system.**



Why?

- in asynchronous systems we cannot distinguish between a failed process and a slow or very distant one
- in synchronous systems we will know if some process crashes = perfect failure detection
- **perfect failure detection = Consensus solvable**
- in asynchronous systems = no failure detection
- **no failure detection = Consensus unsolvable**

What does it mean?

- ☹ we cannot implement TOcast neither VScast in $\mathbb{S}^{\text{Async}}\{\emptyset\}$

What can we do to solve Consensus problem?

- ☹ we can find randomized solutions
- 😊 **we can extend the $\mathbb{S}^{\text{Async}}$ model**

What do we need?

- just failure detection: $\mathbb{S}^{\text{Async}}\{\text{FD}\}$
- is perfect failure detection necessary?

FAILURE DETECTORS

FD = Failure Detector

- rozproszony detektor uszkodzeń = zbiór modułów „wyroczni”
- każda z wyroczni jest przyłączona do jednego procesu
- jej zadaniem jest dostarczanie listy procesów podejrzewanych o uszkodzenie
- wyrocznie mogą być omylne
- błędne podejrzenia nie mogą powstrzymywać poprawnych procesów przed zachowaniem się zgodnie z ich specyfikacją

FD properties:

Completeness

C = eventually every P_f suspected by every P_c

Accuracy

A = no P_c ever suspected by no P_c



FD properties:

Completeness

SC = eventually every P_f suspected by every P_c
Strong Completeness

WC = eventually every P_f suspected by some P_c
Weak Completeness

each P_f may be
suspected by
a different P_c

Accuracy

SA = no P_c ever suspected by no P_c

WA = some P_c never suspected by no P_c

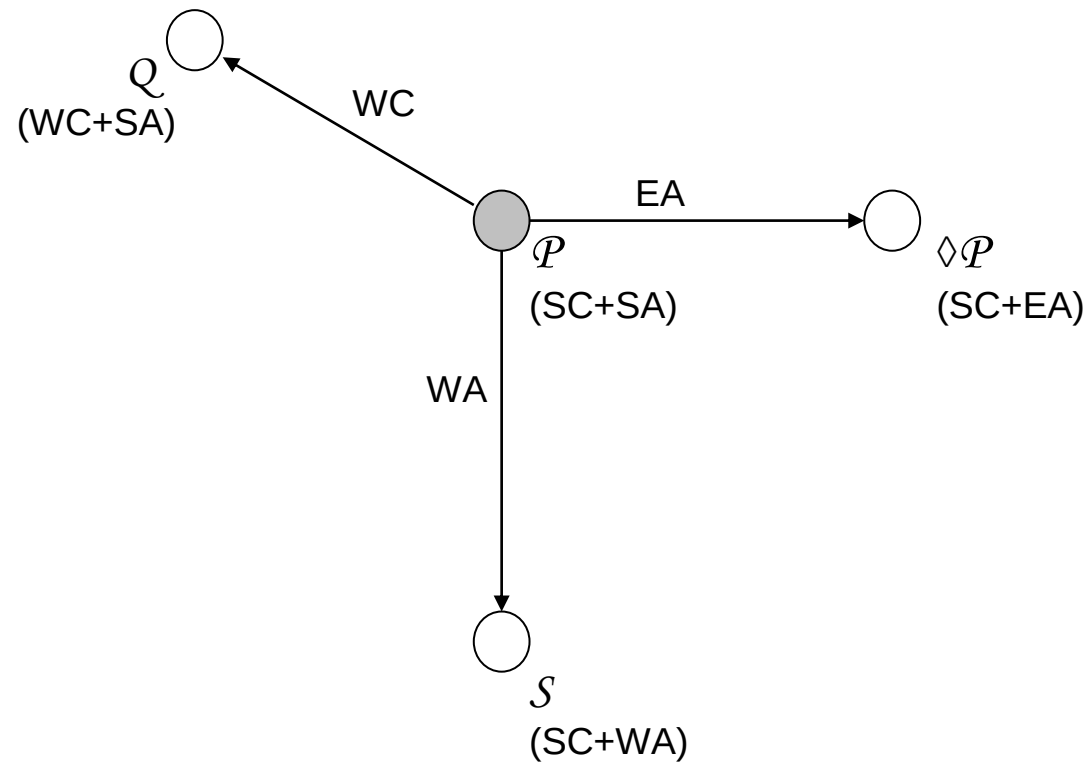
EA = no P_c eventually suspected by no P_c
Eventual Accuracy

EWA = some P_c eventually not suspected by no P_c

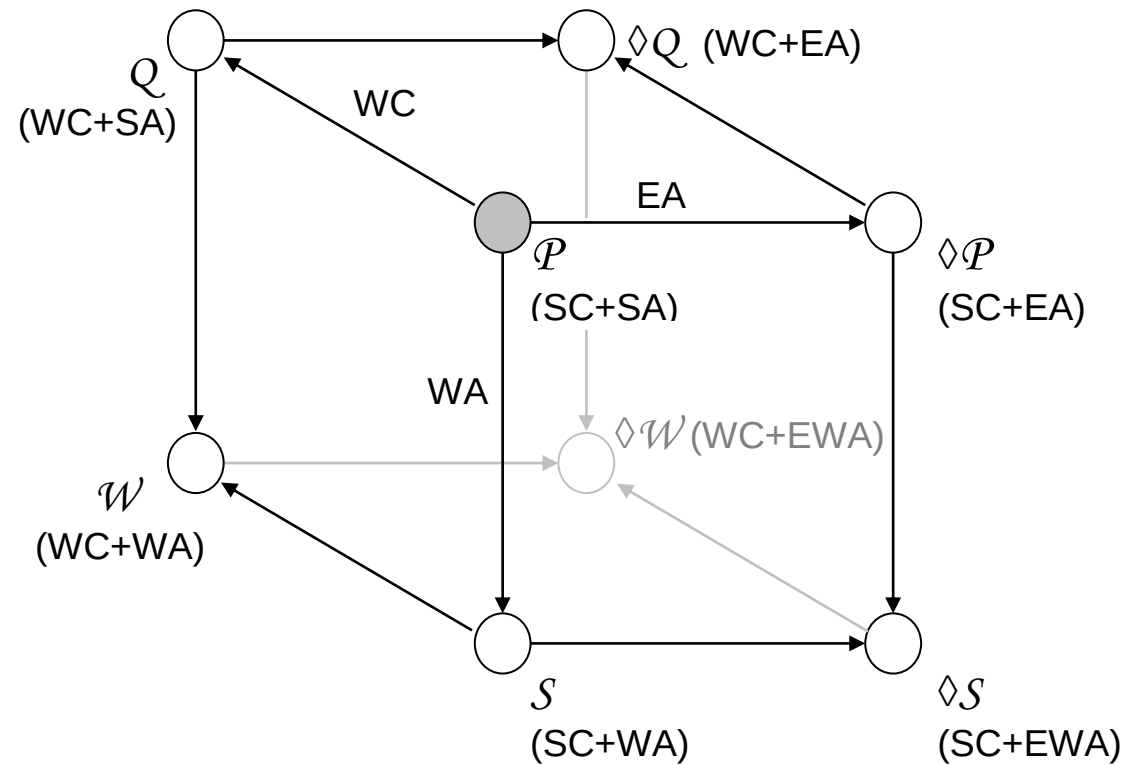
the same P_c
must never be
suspected by all P_c



Relation between FD classes:



Relation between FD classes:



Γ - ACCURATE FAILURE DETECTORS

Rozdział sieci:

- własność dokładności nie jest możliwa do osiągnięcia w przypadku podziału sieci na rozłączne fragmenty (*partitioning*)
- własność Γ -dokładności (ang. *Γ -accuracy*) dotyczy jedynie procesów należących do pewnego podzbioru Γ

Nowe klasy detektorów uszkodzeń:

- silna Γ -dokładność – żaden poprawny proces w zbiorze Γ nie jest podejrzewany przez inny proces z tego zbioru
- słaba Γ -dokładność – pewien poprawny proces (niekoniecznie ze zbioru Γ) nie jest podejrzewany przez żaden proces ze zbioru Γ
- $\Gamma_1 \subset \Gamma_2$: jeśli zachodzi własność Γ -dokładności w zbiorze Γ_2 , to zachodzi też Γ -dokładność w zbiorze Γ_1
- $\mathcal{P}(\Gamma)$, $\diamond \mathcal{P}(\Gamma)$, $\mathcal{Q}(\Gamma)$, $\diamond \mathcal{Q}(\Gamma)$, $\mathcal{S}(\Gamma)$, $\diamond \mathcal{S}(\Gamma)$, $\mathcal{W}(\Gamma)$, $\diamond \mathcal{W}(\Gamma)$

REDUCTION & EQUIVALENCE OF FAILURE DETECTORS

Reduction

Transformation algorithm: $T_{\mathcal{D} \rightarrow \mathcal{D}'}$

Reduction: $\mathcal{D} \succeq \mathcal{D}'$ ($\mathcal{D}$ “emulates” $\mathcal{D}'$)

($\mathcal{D}'$ “is weaker than” $\mathcal{D}$)

example: $SC \succeq WC$

(*obvious: empty transformation*)

Equivalence

$$(\mathcal{D} \succeq \mathcal{D}' \wedge \mathcal{D}' \succeq \mathcal{D}) \Rightarrow \mathcal{D} \cong \mathcal{D}'$$

example: $WC \succeq SC$

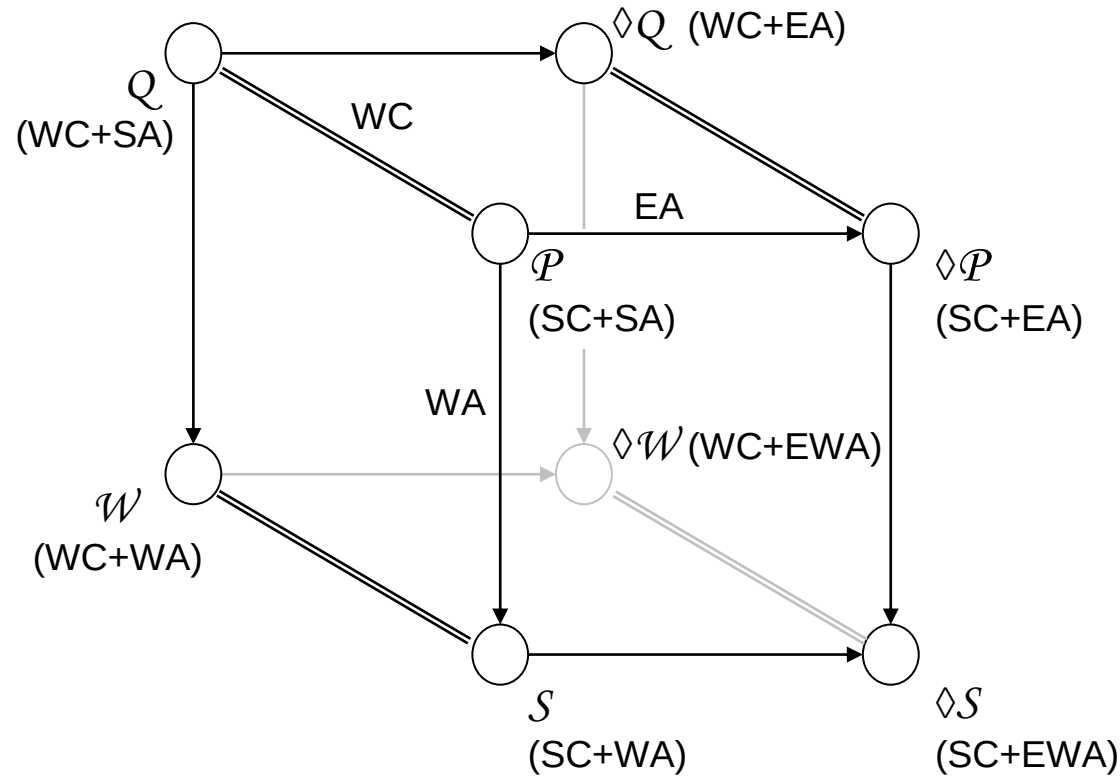
$\Downarrow$

$SC \cong WC$

Transformation algorithm $T_{\mathcal{D} \rightarrow \mathcal{D}'}$ from WC to SC C_i :

```
outputi  $\leftarrow \emptyset$  // outputi will emulate  $\mathcal{D}'_i$ 
|| Task1: repeat forever
{
    suspectedi  $\leftarrow \mathcal{D}_i$  //  $C_i$  queries its local FD module
    send( $i, \text{suspected}_i$ ) to all  $C$ 
}

|| Task2: when receive( $j, \text{suspected}_j$ ) from some  $C_j$ 
{
    outputi  $\leftarrow (\text{output}_i \cup \text{suspected}_j) - P_j$ 
}
```



There holds $P \equiv Q$, $S \equiv W$, $\diamond P \equiv \diamond Q$, $\diamond S \equiv \diamond W$, also $P \succ S$, $P \succ \diamond P$, $S \succ \diamond S$ etc.

Diagonal classes are incomparable: $\diamond P$ and S , $\diamond Q$ and W , $\diamond P$ and W , $\diamond Q$ and S

Moreover $\diamond S(\Gamma) \equiv \diamond S$, but $\diamond W' \succ \diamond W'(\Gamma)$ and $P \succ P(\Gamma)$



EQUIVALENCE $C \cong \text{TOcast}$

Reduction $\text{TOcast} \succeq C$ $\rightarrow$ each P_c decides first TO-delivered value

*Reduction $C \succeq \text{TOcast}$ $\rightarrow$ **periodic** Consensus on subsets of
messages yet to deliver
a priori **deterministic** order of message
delivery in a subset*

*Reduction $C \succeq \text{VScast}$ $\rightarrow$ **periodic** Consensus on subsets of
delivered messages and composition of
next view v_{i+1}
a priori **deterministic** order of delivery for
messages sent in view v_i*



SOLVING TOCAST USING CONSENSUS [1]

C_i :

```
init:
    R_delivered =  $\emptyset$ 
    TO_delivered =  $\emptyset$ 
    k = 0

TOcast(m):
    RBcast(m)

TOdeliver() occurs as follows:
    || when Rdeliver(m)
        R_delivered = R_delivered  $\cup$  {m}

    || when R_delivered - TO_delivered  $\neq \emptyset$ 
        k++
        TO_undelivered = R_delivered - TO_delivered
        propose(k, TO_undelivered)
        wait until decide(k, msgSetk)
        deliver all msg from msgSetk in deterministic order
        TO_delivered = TO_delivered  $\cup$  msgSetk
```



SOLVING TOCAST USING CONSENSUS [1]

 C_i :

```
init:
    R_delivered =  $\emptyset$ 
    TO_delivered =  $\emptyset$ 
    k = 0

TOcast(m):
    RBCast(m)

TOdeliver() occurs as follows:
    || when Rdeliver(m)
        R_delivered = R_delivered  $\cup$  {m}

    || when R_delivered - TO_delivered  $\neq \emptyset$ 
        k++
        TO_undelivered = R_delivered - TO_delivered
        propose(k, TO_undelivered)
        wait until decide(k, msgSetk)
        TO_deliverk = msgSetk - TO_delivered
        deliver all msg from TO_deliverk in deterministic order
        TO_delivered = TO_delivered  $\cup$  TO_deliverk
```

SOLVING CONSENSUS USING FD [2]

FD $\in \mathcal{S}$:

- $\mathcal{S} = \text{SC} + \text{WA} \Rightarrow$ co najmniej jeden P_c nigdy nie podejrzewany („autorytet”)
- N rund:
 - $N-1$ propozycji
 - ogłoszenie ostatecznego zbioru propozycji
 - „wyliczana” część wspólna (*de facto* decyduje autorytet)
 - decyzja = pierwsza pozostała propozycja
- $f < N$
- eventual reliable channels

Remember: $\mathcal{S} \cong \mathcal{W}$

FD $\in \Diamond S$:

- $\Diamond S = SC + EWA \Rightarrow$ na początku każdy proces może być podejrzewany (nie ma autorytetu)
- liczba rund nie jest ograniczona (do skutku):
 - każdy z procesów kolejno staje się koordynatorem rundy (*rotating coordinator paradigm*)
 - koordynator czeka na zebranie quorum Q_c
 - decyzja – rozgłaszana przez RBcast
- $f < \left\lceil \frac{N}{2} \right\rceil$, gdyż $Q_c = \left\lfloor \frac{N}{2} \right\rfloor + 1$ (większość)
- eventual reliable channels

Remember: $\Diamond S \cong \Diamond \mathcal{W}$



Uniform Consensus

Any protocol solving Consensus using $\diamond S$ also solves Uniform Consensus.

OTHER PROBLEMS

Are there any other well-known problems whose solutions require FD stronger than $\diamond S$?

TRBcast (*Terminating Reliable Broadcast*):

Termination: *every P_c eventually delivers exactly one message*

Validity: *if P_s broadcasts m and remains correct $\Rightarrow$ it eventually delivers m*

Agreement: *if some P_c delivers $m \Rightarrow$ all P_c deliver m*

Integrity: *m was broadcast by P_s and is delivered at most once*

There is a distinguished process P_s that is supposed to broadcast a single message. TRBcast problem is similar to RBcast, except that it requires every P_c to always deliver a message – even if P_s is faulty and, say, crashes before broadcasting. In that case we allow processes to deliver a special message m_F (i.e. *null*) that was not actually broadcast (and proves that P_s has failed). By convention, we set $\text{sender}(m_F) = P_s$.

(This problem is in fact equivalent to Byzantine Agreement)



Theorems:

1. TRB can be solved in asynchronous systems with any number of crashes using $\mathcal{P}$.
2. TRB cannot be solved in asynchronous systems using either $\diamond\mathcal{P}$, $\mathcal{S}$ or $\diamond\mathcal{S}$, even in assumption of at most one crash failure.

In fact, $\mathcal{P}$ is the weakest FD required to solve repeated instances of TRB
(multiple instances of each process as $\mathbf{P_s}$)



Comment:

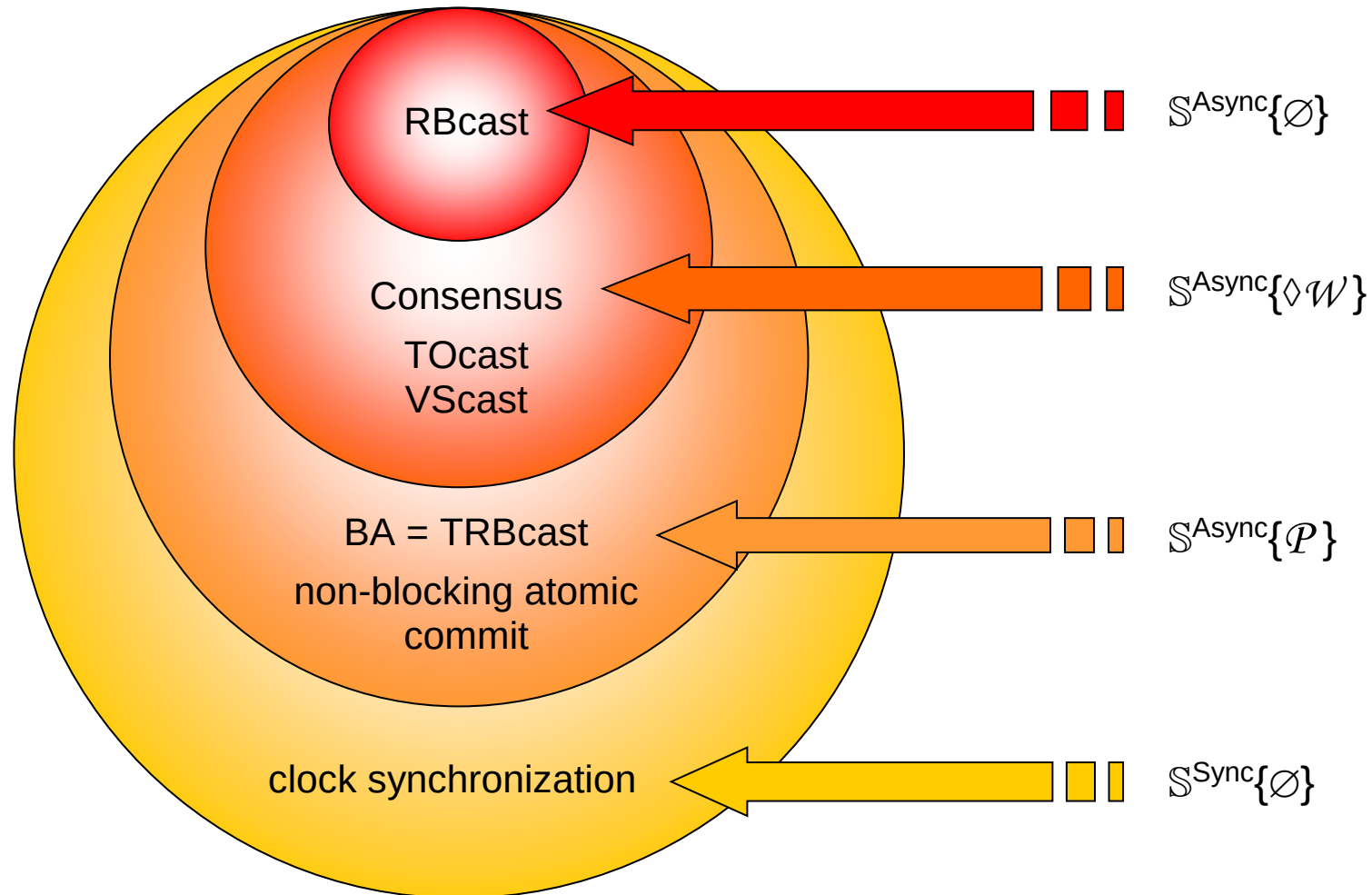
The key difference between TRB and RBcast is that TRB requires eventual delivery of some message – even if the sender fails just when about to broadcast (in this case, a *null* message gets sent).

Hence, TRB requires the recognition of the failure (i.e. failure detection) as opposed to no message getting sent.

So, it is equal to the ability of distinguishing between a slow process and a failed process.



Solvability of problems:



Failure Detectors are a nice tool

Advantages:

- natural – genuine extension of the asynchronous system model $\mathcal{S}^{\text{Async}}$
- minimal – minimum additional information sufficient to solve Consensus: $\mathcal{S}^{\text{Async}}\{\text{FD}\}$
- straightforward – easy to understand and use
- portable – can be used to solve a huge range of problems (group membership, election, atomic commit, ...)
- efficient – for small f

Drawback:

- complicated algorithms – must deal with FD's mistaken suspicions

Failure Detectors are a theoretical tool

Not-implementable in $\mathcal{S}^{\text{Async}}$:

- FD properties must hold **forever** (event. starting from some unknown but finite time)

In practice – implementation close to FD properties:

- must hold **long enough** (i.e. to solve the problem)

indulgent algorithms

An *indulgent* algorithm based on FD terminates and produces a correct output if the FD behaves accordingly to its specification. If the FD does not satisfy its specification, the algorithm may not terminate, but if it does, it always produces a correct output.

- we assume existence of *stable periods* during which the FD properties holds
- during stable periods consensus and replication (active and passive) are possible



BIBLIOGRAPHY

- [1]  M. J. Fischer, N. A. Lynch, M. S. Paterson, “Impossibility of distributed consensus with one faulty process”. Journal of the ACM no. 32, 1985, pp.374-382.
- [2]  T.D. Chandra, S. Toueg, “Unreliable Failure Detectors for Reliable Distributed Systems”. In Proc. 10th ACM Symposium on Principles of Distributed Computing, ACM Press, 1991.
- [3]  J. Kobusiński, A. Szajkowska, M. Szychowiak, „Detektory uszkodzeń a niezawodna komunikacja grupowa”, VI Konferencja POLMAN '99, 1999.
(<https://www.cs.put.poznan.pl/mszychowiak/publications/Polman99-FD.ps.gz>)
- [4]  J. Brzeziński, J. Kobusiński, “A Survey of Software Failure Detector Protocols”, Foundations of Computing and Decision Sciences, Vol. 28, No.2, 2003.
- [5]  G. Tel, “Introduction to Distributed Algorithms”, Cambridge University Press, 2000, ch. 16.
- [6]  A. D. Kshemkalyani, M. Singhal, “Distributed Computing – Principles, Algorithms, and Systems”, Cambridge University Press, 2008, ch. 15.
- [7]  M. Raynal, “Communication and Agreement Abstractions for Fault-Tolerant Asynchronous Distributed Systems”, Morgan & Claypool Pub., 2010.